



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Smart Electric Vehicle Charging Network Management Platform

Architecture Design Assignment

Submitted in the partial fulfilment for the Course of

CSA1016 – SOFTWARE ENGINEERING

to the award of the degree of

**BACHELOR OF ENGINEERING IN
ARTIFICIAL INTELLIGENCE AND DATA SCIENCE**

Submitted by

R. R. RAKSHITH (192324156)

Under the Supervision of

Dr. K. Anitha Davamani

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

JUL 2026



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105



DECLARATION

I, R. R. Rakshith, of the Department of Artificial Intelligence & Data Science, Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Case study work entitled **Smart Electric Vehicle Charging Network Management Platform** is the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place: Chennai

Date: 06/08/2026

Signature of the Students with Names

R R RAKSHITH (192324156)

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, Dr. N.M. Veeraiyan, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, Dr. Deepak Nallaswamy Veeraiyan, and our Vice-Chancellor, Dr. S. Suresh Kumar, for their visionary leadership and moral support during the course of this project.

We are truly grateful to our Director, Dr. Ramya Deepak, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, Dr. B. Ramesh for granting us access to the institute's facilities and encouraging us throughout the process. We sincerely thank our Head of the Department, **Dr. Anusuya** for her continuous support, valuable guidance, and constant motivation.

We are especially indebted to our guide, **Dr. K. Anitha Davamani** for her creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members (Internal and External), and the entire faculty team for their constructive feedback and valuable inputs that helped improve the quality of our work. Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

Signature With Student Name

R R RAKSHITH (192324156)

TABLE OF CONTENTS

S.NO	TOPIC	PAGE NO.
1	SYSTEM REQUIREMENTS ANALYSIS	3
2	SELECTION OF SOFTWARE ARCHITECTURE	4-5
3	SOFTWARE ARCHITECTURE DIAGRAM	6
4	COMPONENTS AND THEIR INTERACTIONS	7-8
5	QUALITY ATTRIBUTES	9-10
6	ARCHITECTURAL JUSTIFICATION AND TRADE-OFFS	11
7	UML DESIGN DIAGRAMS (CLASS, USE CASE & ACTIVITY)	12-14

1. System Requirements Analysis

1.1 System Objectives

The Smart EV Charging Network Management Platform must connect EV owners, station operators, and administrators through a single digital system that provides real-time station discovery, advance slot reservation, automated charging-session handling, digital payment, predictive maintenance, and network analytics. The architecture must be designed so that these capabilities can be delivered reliably as the number of stations, users, and transactions grows over time.

1.2 Functional Requirements Influencing the Architecture

- Real-time synchronization of charging station status between IoT hardware and the cloud platform — requires a low-latency, message-based communication mechanism rather than pure request/response polling.
- Concurrent slot reservation across many stations — requires transactional consistency at the data layer to prevent double-booking.
- Independent evolution of features (booking, payment, analytics) — requires the system to be decomposed into independently deployable units rather than a single monolith.
- Integration with external systems (payment gateway, mapping service) — requires well-defined API boundaries and gateway-based routing.
- Fault detection from streaming sensor data — requires an event-driven processing path in addition to standard CRUD APIs.

1.3 Non-Functional Requirements Influencing the Architecture

Quality Attribute	Architectural Implication
Scalability	The system must scale horizontally as stations and users grow into the thousands/tens of thousands — favours stateless, independently scalable services.
Performance	Station search and availability must respond within ~2 seconds — favours caching and asynchronous telemetry ingestion rather than synchronous polling of every station.
Reliability	Payment and session data must not be lost even during partial outages — favours message queuing and retry mechanisms over direct synchronous-only calls.
Security	Payment and personal data must be protected — favours a centralized API gateway for

	authentication/authorization rather than each service handling security independently.
Maintainability	Different teams may work on booking, payment, and analytics independently — favours loosely coupled components with clear API contracts.
Availability	The platform must remain available close to 24/7 — favours redundancy and the ability to deploy/restart individual services without full-system downtime.

2. Selection of Software Architecture

2.1 Candidate Architectural Styles Considered

Architectural Style	Suitability for This System
Monolithic Layered Architecture	Simple to build initially, but a single deployable unit makes independent scaling of booking vs. payment vs. analytics impossible, and any change requires redeploying the entire system — not suitable for a growing, multi-team platform.
Pure Client-Server	Too coarse-grained; does not naturally support the mix of synchronous booking requests and asynchronous IoT telemetry streams the platform needs.
Service-Oriented Architecture (SOA)	Conceptually close to what is needed, but its typical heavyweight enterprise service bus (ESB) middleware adds unnecessary complexity/latency compared to lightweight REST-based microservices for this scale of system.
Serverless (Functions-as-a-Service)	Attractive for bursty, event-driven workloads (e.g., fault-alert processing) but less suitable as the sole style for stateful, latency-sensitive booking transactions due to cold-start latency.
Microservices + Event-Driven (Hybrid) — SELECTED	Allows each business capability (auth, booking, session, payment, analytics, maintenance) to be developed, deployed, and scaled independently, while an event-driven messaging layer cleanly handles the high-volume, asynchronous IoT telemetry stream separately from synchronous user-facing APIs.

2.2 Selected Architecture: Hybrid Microservices with Event-Driven Components

The proposed system adopts a Hybrid Microservices Architecture: business capabilities are decomposed into independent microservices exposed through a central API Gateway (synchronous REST/HTTPS communication for user-facing operations), combined with an Event-Driven Architecture (EDA) layer built around a message broker (MQTT + Kafka-style event streaming) for high-volume, asynchronous IoT telemetry and fault-event processing.

2.3 Justification

- Independent scalability: the Booking Service and Session Service (high traffic) can be scaled independently of the Analytics Service (lower, batch-oriented traffic), which a monolith cannot do.
- Independent deployability: new features (e.g., a new payment method) can be deployed to the Payment Service alone without redeploying the entire platform, directly supporting continuous delivery under the Agile/Scrum process used for this project.
- Natural fit for IoT data: charging stations generate a continuous stream of telemetry that is naturally asynchronous — an event-driven messaging layer decouples station hardware from application services and prevents telemetry spikes from overwhelming synchronous APIs.
- Fault isolation: if the Analytics Service fails, EV owners can still search, book, and charge — the failure is contained rather than bringing down the whole platform, directly supporting the Reliability and Availability requirements.
- Alignment with team structure: separate microservices allow separate Scrum teams to own separate services, consistent with the Agile Sprint Planning already carried out for this project.

3. Software Architecture Diagram

The diagram below shows the complete hybrid architecture: a client presentation layer, an API Gateway layer for synchronous routing and security, a microservices business-logic layer, an event-driven messaging layer for asynchronous IoT communication, a polyglot data layer, the field/IoT layer, and external systems (payment gateway, mapping service).

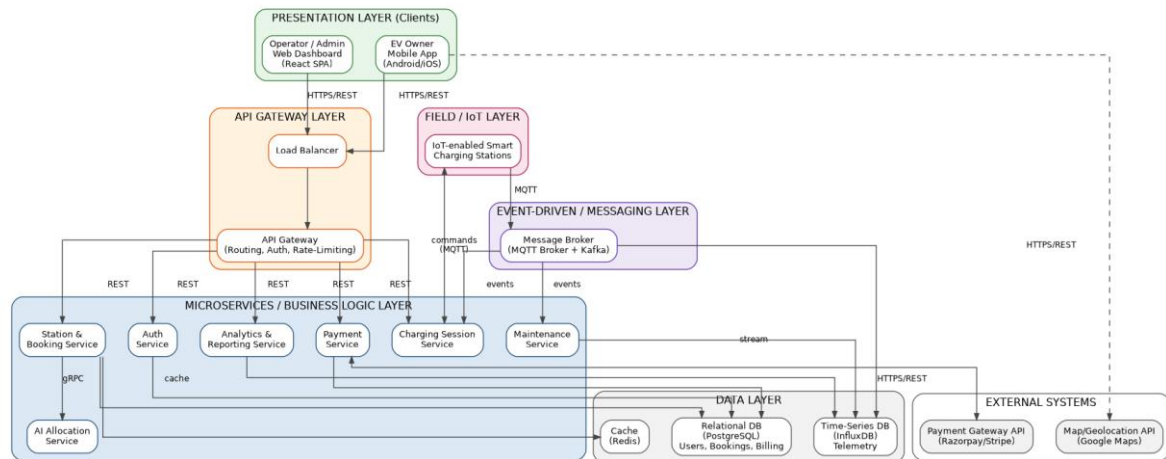


Figure 3.1: Hybrid Microservices + Event-Driven Architecture Diagram

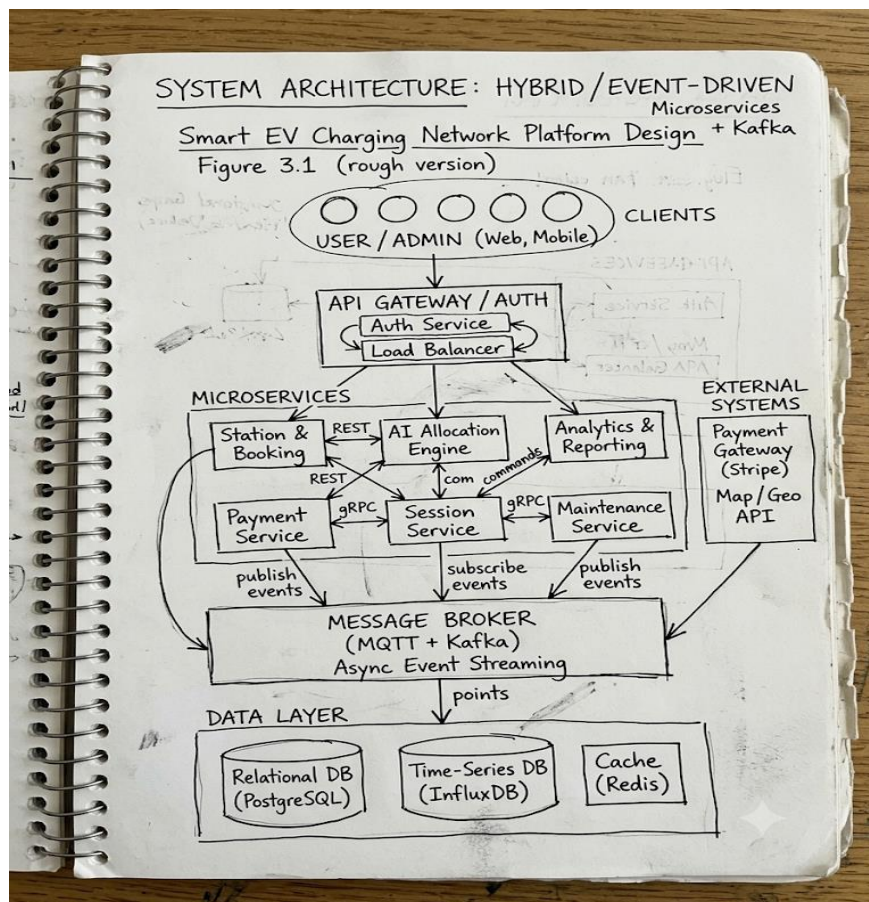


Figure 3.2: Hand written Architecture Diagram

4. Components and Their Interactions

4.1 Component Responsibilities

Component	Responsibility
API Gateway	Single entry point for all client requests; handles authentication, request routing, and rate-limiting.
Load Balancer	Distributes incoming traffic across multiple instances of the API Gateway/services for high availability.
Auth Service	Manages user registration, login, and token issuance/validation.
Station & Booking Service	Handles station search, availability queries, and slot reservation logic.
AI Allocation Service	Recommends the optimal station/slot based on location, live occupancy, and demand prediction.
Charging Session Service	Manages session authentication, start/stop commands, and energy-usage metering.
Payment Service	Calculates cost, integrates with the external payment gateway, and generates invoices.
Maintenance Service	Consumes fault events from telemetry and raises alerts to operators/technicians.
Analytics & Reporting Service	Aggregates usage, revenue, and telemetry data into reports and dashboards.
Message Broker (MQTT + Kafka)	Decouples IoT stations from backend services via publish/subscribe event streaming.
Relational DB (PostgreSQL)	Stores structured transactional data — users, bookings, and billing records.
Time-Series DB (InfluxDB)	Stores high-volume telemetry data optimized for time-based queries and analytics.
Cache (Redis)	Reduces database load and latency for frequently accessed data such as live station availability.

4.2 Communication Mechanisms

The architecture uses two distinct communication patterns depending on the nature of the interaction: synchronous REST/HTTPS calls for user-initiated, request/response operations, and asynchronous publish/subscribe messaging for high-frequency IoT telemetry and fault events. This separation keeps user-facing latency low while still processing continuous sensor data efficiently.

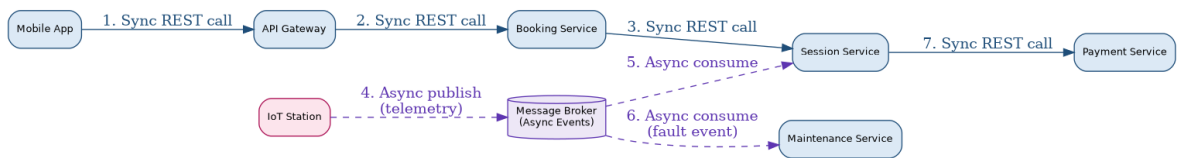


Figure 4.1: Synchronous vs Asynchronous Communication Patterns

4.3 End-to-End Workflow

The numbered workflow below illustrates how a typical booking-to-charging request flows through the architecture's components, from the initial request by the EV owner to the final analytics update.

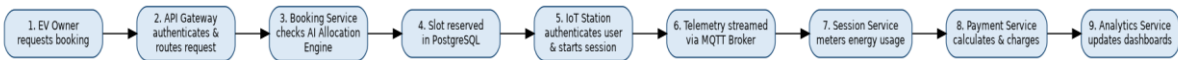


Figure 4.2: End-to-End Request Workflow Through the Architecture

5. Quality Attributes

5.1 Scalability

Each microservice is stateless and independently deployable behind the load balancer, allowing horizontal scaling — additional instances (pods) of high-traffic services such as Booking and Session can be added automatically under load, as illustrated in the scalability/deployment view below, without needing to scale lower-traffic services such as Analytics.

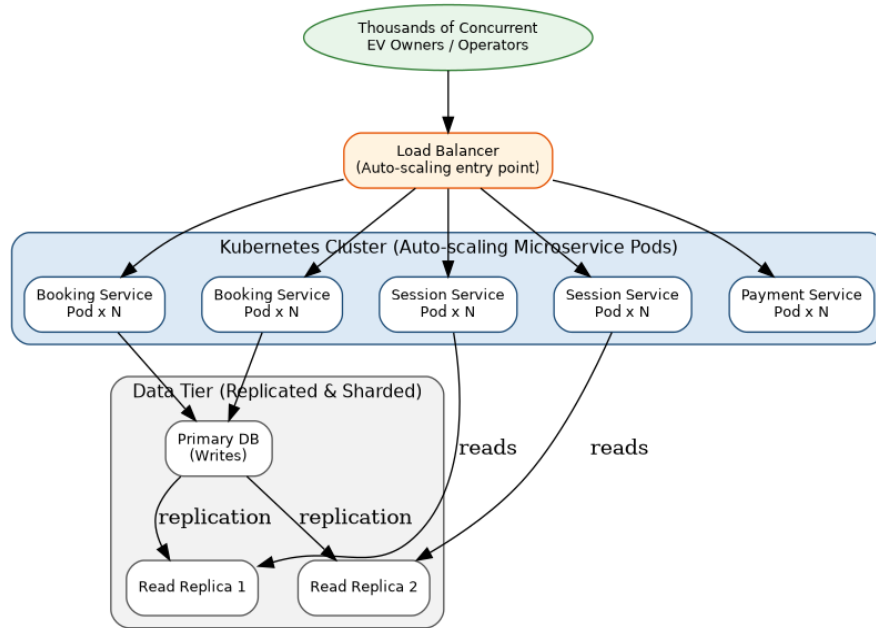


Figure 5.1: Scalability and Deployment View (Auto-scaling & DB Replication)

5.2 Security

All external traffic passes through the API Gateway, which centralizes authentication and authorization, so individual microservices do not each need to re-implement security logic. Data is encrypted in transit (HTTPS/TLS) and at rest, and the Payment Service isolates sensitive financial data from the rest of the system, communicating with the external payment gateway over a secured channel.

5.3 Performance

Frequently accessed data — such as live station availability — is served from the Redis cache rather than querying the relational database on every request, keeping response times low. Asynchronous ingestion of IoT telemetry via the message broker prevents high-frequency sensor updates from adding load to the synchronous, user-facing request path.

5.4 Reliability

The event-driven messaging layer provides durable, at-least-once delivery of telemetry and fault events, so temporary service outages do not result in lost data — messages are retained in the broker until successfully processed. Database replication (primary plus read replicas) further protects against data loss and read-path failures.

5.5 Maintainability

Because each microservice owns a single, well-defined responsibility and communicates only through defined API/event contracts, individual services can be modified, tested, and redeployed independently — a change to the Payment Service, for example, does not require retesting or redeploying the Booking or Analytics services.

5.6 Availability

Redundant instances behind a load balancer mean the platform can continue serving requests even if an individual service instance fails. Fault isolation between microservices ensures that a failure in one component (e.g., Analytics) does not cascade into a full-platform outage, supporting the target of near-continuous availability.

6. Architectural Justification and Trade-offs

6.1 Rationale Summary

The hybrid microservices and event-driven architecture was chosen because it directly satisfies the platform's core non-functional demands — independent scalability, fault isolation, and efficient handling of high-volume asynchronous IoT data — while remaining compatible with the Agile, team-based delivery approach already established for this project's sprint planning.

6.2 Trade-offs Considered

- Operational complexity vs flexibility: microservices introduce more moving parts (service discovery, inter-service networking, distributed monitoring) than a monolith, but this cost is accepted in exchange for independent scalability and deployability, which are essential at the platform's target scale.
- Eventual consistency vs strict consistency: the event-driven telemetry path favours eventual consistency (a brief delay before a station's status updates network-wide) in exchange for higher throughput and resilience; this is an acceptable trade-off since a few seconds of telemetry lag does not materially affect the user experience, whereas booking/payment transactions retain strict consistency in the relational database.
- Infrastructure cost vs reliability: running redundant service instances and database replicas increases infrastructure cost compared to a single-instance deployment, but this is justified by the platform's availability requirement given its role in daily commuter mobility.

6.3 Support for Future Enhancements

- New capabilities (e.g., Vehicle-to-Grid support, dynamic pricing, route-based trip planning) can be added as new independent microservices without modifying existing ones.
- The event-driven layer can absorb new IoT data types (e.g., battery health telemetry) by adding new event topics/consumers without changing the synchronous API surface.
- The API Gateway provides a natural integration point for future external partners (e.g., fleet management systems) without exposing internal service boundaries.
- Because services are independently deployable, the system can adopt new technology (e.g., a new database for a specific service) incrementally rather than requiring a full-platform migration.

In summary, the selected hybrid microservices and event-driven architecture provides the scalability, reliability, security, maintainability, and availability the Smart EV Charging Network Management Platform requires today, while remaining flexible enough to support the system's long-term growth and evolution.

7. UML Design Diagrams (Class, Use Case & Activity)

In addition to the architecture-level diagrams presented in the earlier sections, the following UML design diagrams were modelled using Umbrello UML Modeller to capture the static structure and dynamic behaviour of the Smart EV Charging Network Management Platform at the object-oriented design level.

7.1 Class Diagram

The class diagram below models the core domain entities of the platform and their relationships, including inheritance between the generic User class and its EVOwner/Admin specializations, and associations between Booking, ChargingStation, ChargingSession, and Payment.

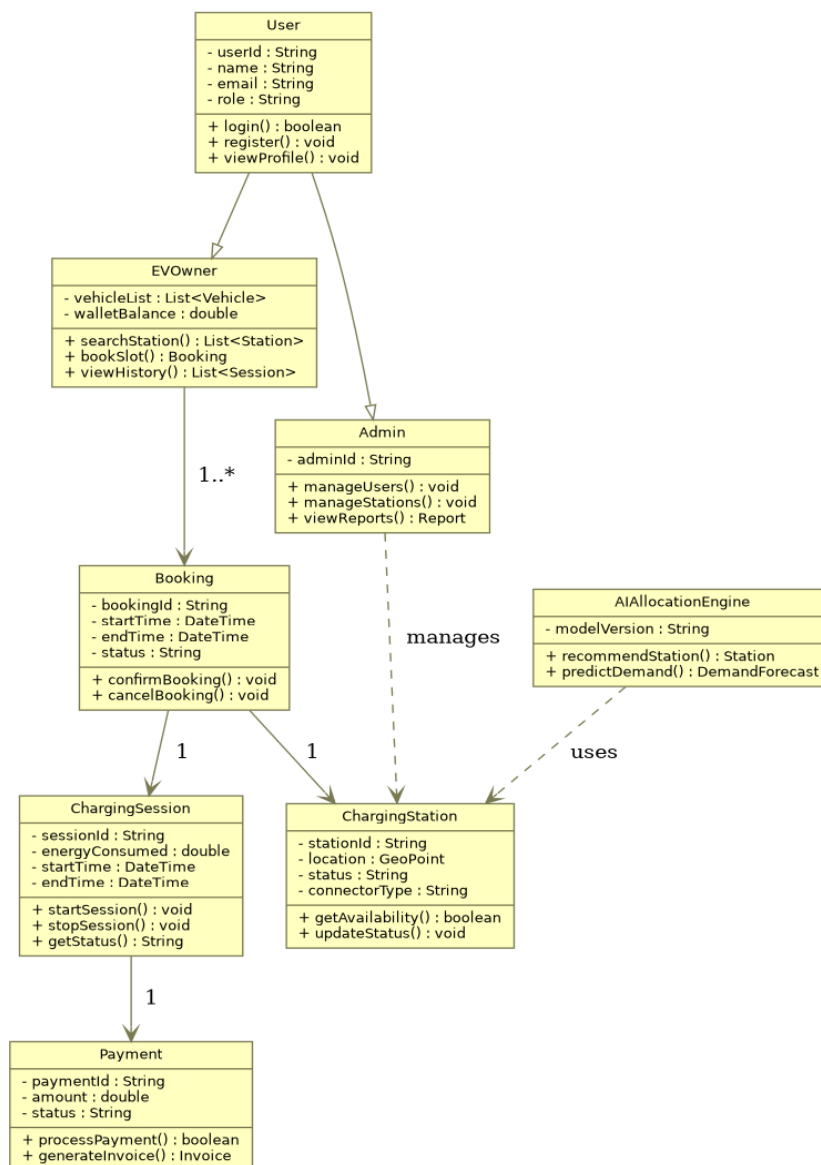


Figure 7.1: UML Class Diagram (Umbrello)

7.2 Use Case Diagram

The use case diagram below captures the interactions between the EV Owner, Station Operator, and System Admin actors and the platform's core use cases, including the external Payment Gateway invoked via an «include» relationship from Make Payment.

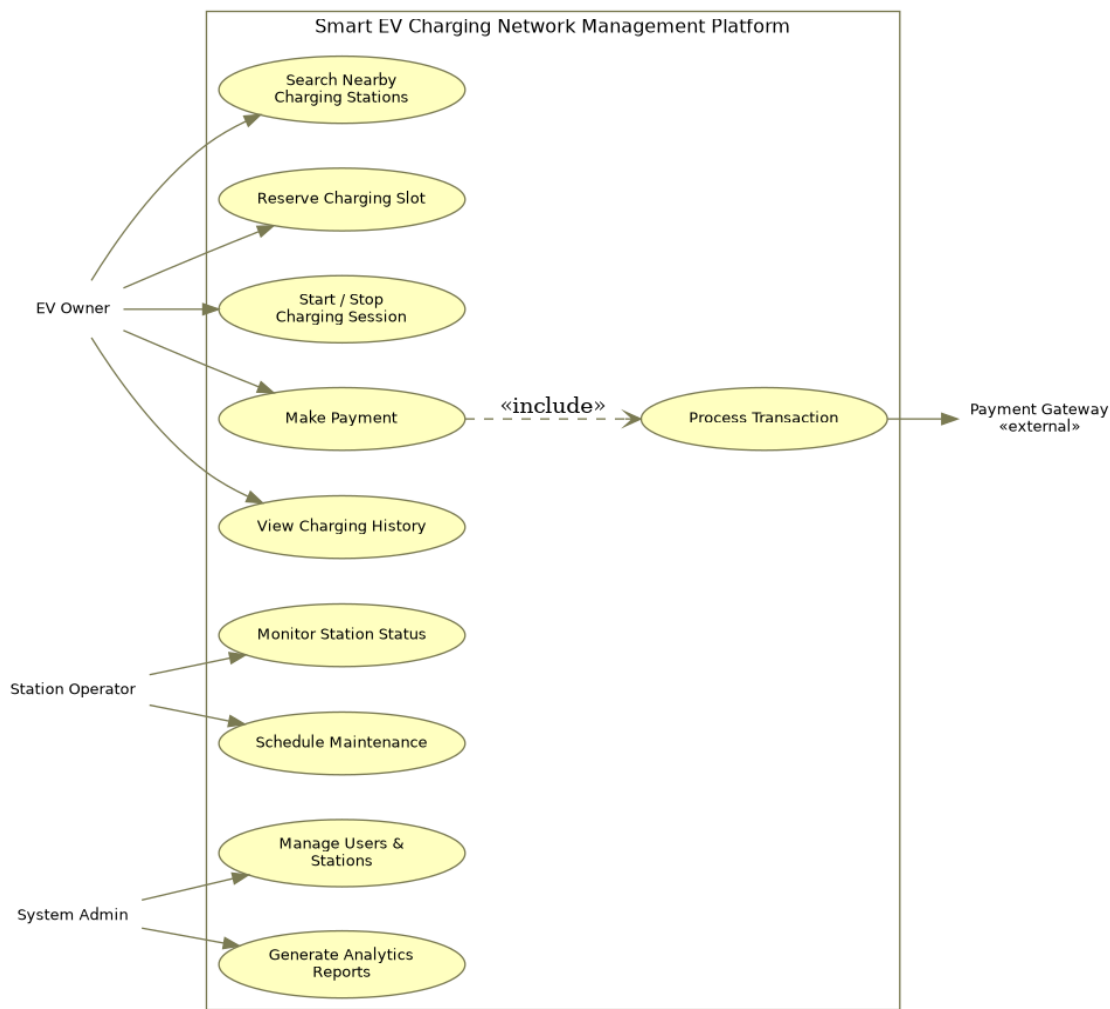


Figure 7.2: UML Use Case Diagram (Umbrello)

7.3 Activity Diagram

The activity diagram below traces the end-to-end workflow an EV owner follows, from logging into the mobile app through to searching, reserving, charging, and paying — including the decision points for slot availability and charging completion.

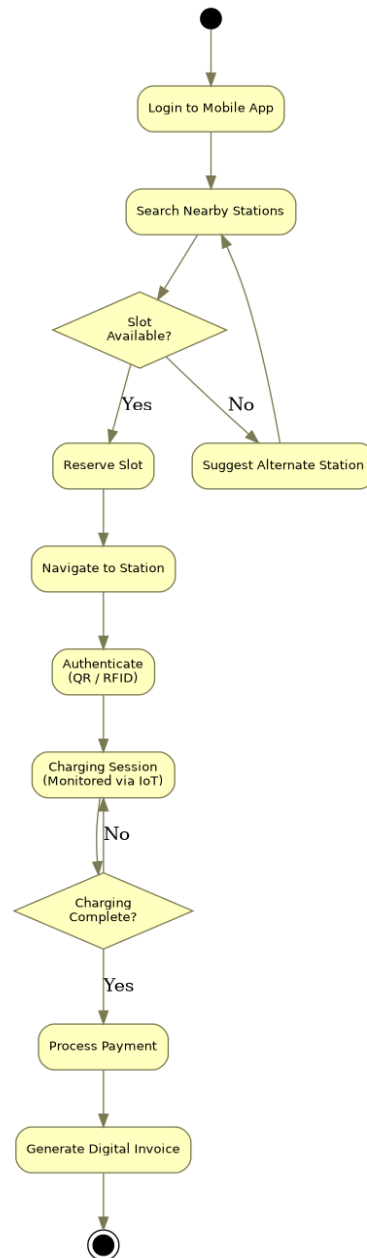


Figure 7.3: UML Activity Diagram (Umbrello) — EV Charging Workflow